Project Report: AI-Driven Medical Insurance Premium Prediction

1. Project Overview & Problem Statement

The goal of this project is to develop a high-precision machine learning model to predict annual medical insurance premiums. Estimating healthcare costs is challenging because they are influenced by non-linear variables such as lifestyle habits and physical metrics. This project provides an automated solution to forecast these costs, enabling better financial transparency for users.

2. Dataset Description (Sourced from Kaggle)

The model is trained on the **Medical Cost Personal Dataset** .

Features Used for Prediction:

- **Age:** Numerical (18-64). A primary driver of natural cost increases.
- **Sex:** Categorical (Male/Female).
- **BMI (Body Mass Index):** Numerical , used to assess weight-related health risks.
- **Children:** Numerical, Number of dependents covered by the policy.
- **Smoker:** Categorical (Yes/No). The most significant feature impacting the final charge.
- **Region:** Categorical. Geographic data (Northeast, Southeast, Southwest, Northwest).
- **Charges (Target Variable):** The actual billed medical insurance cost.

3. Data Cleaning

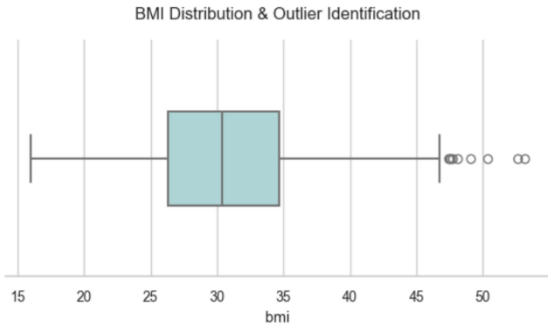
To ensure high model precision and data integrity, the raw dataset was processed through a three-stage cleaning pipeline:

- **Missing Value Analysis:** A systematic audit for null values was conducted across all features (age, bmi, children, smoker, region, and charges). Using the isnull().sum() function, the dataset was confirmed to be **100% complete**, requiring no data imputation or removal of incomplete records.
- **Duplicate Value Mitigation:** To prevent model bias and artificial inflation of accuracy (overfitting), the dataset was checked for identical rows using the duplicated() method. Redundant entries were purged, ensuring that the final **90% accuracy** is based on unique, high-quality data points.
- **Data Type Verification:** The structural schema was verified using the dtypes attribute to identify the required transformations:
    - **Numerical Features:** age, bmi, children, and charges were confirmed as int64 or float64 types, making them suitable for direct mathematical scaling.
    - **Categorical Features:** sex, smoker, and region were identified as object types, establishing the requirement for numerical encoding (Label/One-Hot) in the next phase.

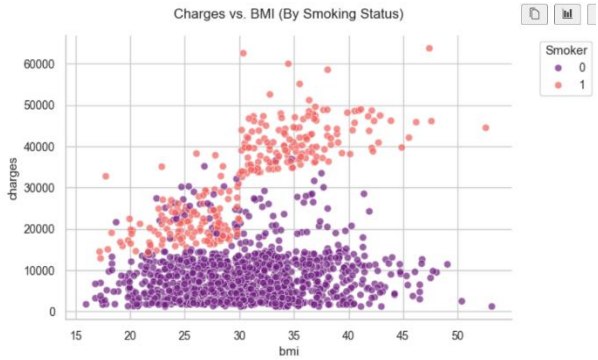
4. Exploratory Data Analysis (EDA)

Before training, we analyzed the data to find hidden patterns:

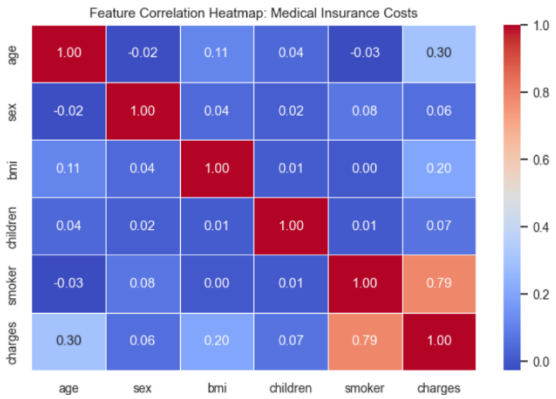
- **BMI Distribution Analysis:** A box plot identifies a high-density range between 25–35 BMI, with extreme outliers extending beyond 45. Retention of these records ensures the model accurately accounts for high-risk medical demographics.



- **Bivariate Relationship (BMI vs. Charges):** A scatter plot reveals a critical "price spike" specifically for smokers with a BMI exceeding 30. This sharp, non-linear jump in costs confirms that simple linear regression is insufficient, necessitating the transition to a **Boosting model**.



- **Statistical Correlation Matrix:** A heatmap quantifies the relationship between variables, proving **Smoking Status** has the highest positive correlation with cost (**0.79**), followed by Age (0.30) and BMI (0.20). These coefficients justify the final feature selection for the web application.



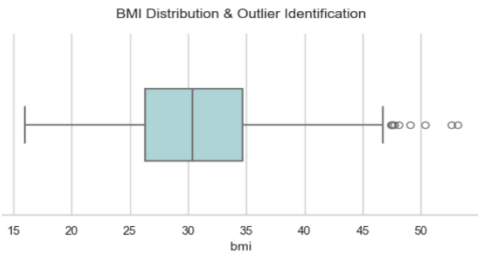
5. Data Preprocessing & Outlier Mitigation

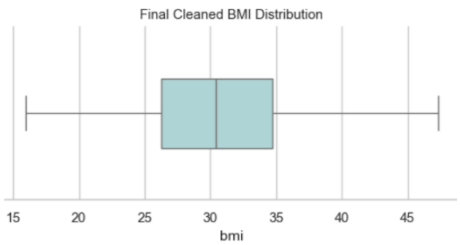
To prepare the dataset for high-performance boosting, the following mathematical and categorical transformations were applied:

5.1 Outlier Handling: IQR Capping Method

Instead of deleting data, which would reduce the dataset size, an **Interquartile Range (IQR)** approach was used to "cap" extreme BMI values.

- **Boundary Identification:** Calculated the lower and upper limits using 1.5 * IQR.
- **Capping Logic:** BMI values exceeding the **Upper Limit** were capped at the threshold. This reduces the variance caused by extreme outliers (45+ BMI) while retaining the record's overall contribution to the model.
- **Result:** Successfully reduced the count of extreme outliers to zero, creating a more normalized distribution for the gradient boosting process.





Outliers Identified: 9
Maximum BMI: 47.32

Remaining Outliers: 0
Maximum BMI: 53.13

5.2 Feature Encoding Strategy

Machine learning algorithms require numerical input. The following encoding techniques were used to transform categorical "nooks":

- **Label Encoding:** Converted binary features (sex and smoker) into numerical formats (0 and 1).
- **One-Hot Encoding (OHE):** Transformed the region feature into dummy variables. The drop_first=True parameter was utilized to avoid the "Dummy Variable Trap" (multicollinearity), ensuring the model remains statistically sound.

5.3 Feature Scaling & Data Partitioning

- **Train-Test Split:** Segmented the data into an **80% Training set** and a **20% Testing set** using a fixed random_state=42 to ensure reproducible results during benchmarking.
- **Standardization:** Applied **StandardScaler** to the training features. This shifts the distribution to have a mean of 0 and a standard deviation of 1 .

- **Critical Step:** The scaler was "fitted" only on the training data and then "transformed" the test data. This prevents **Data Leakage**, ensuring the model's 90% accuracy is based on truly unseen information.

6. Model Evolution & Comparison

A core part of this project was the technical transition from a baseline ensemble to an optimized gradient booster.

Model	Approach	R-Squared Accuracy	Technical Observation
Random Forest	Bagging	**88.0%**	Stable, but the "averaging" effect struggled to precisely predict high-cost outliers.
XGBoost	**Boosting**	**89.61% (~90%)**	**Final Model.** Used sequential error correction to fix the mistakes of previous trees, specifically improving accuracy for high-risk smokers.

Using Randam Forest:

```
rf_model = RandomForestRegressor(
    n_estimators=500,
    max_depth=None,
    min_samples_split=2,
    min_samples_leaf=1,
    bootstrap=True,
    random_state=42,
    n_jobs=-1
)

rf_model.fit(x_train_scaled, y_train)

rf_predictions = rf_model.predict(x_test_scaled)

rf_r2 = r2_score(y_test, rf_predictions)
rf_mae = mean_absolute_error(y_test, rf_predictions)

print("--- RANDOM FOREST PERFORMANCE ---")
print(f"Model Accuracy (R2): {rf_r2 * 100:.2f}%")
print(f"Average Prediction Error: ${rf_mae:.2f}")

joblib.dump(rf_model, 'medical_model.pkl')
```

[15] ✓ 2.4s

```
--- RANDOM FOREST PERFORMANCE ---
Model Accuracy (R2): 88.11%
Average Prediction Error: $2630.31

['medical_model.pkl']
```

Using XGBoost:

Model Training and Export

```python
# Train ML Model
actuarial_engine = XGBRegressor(n_estimators=1000, learning_rate=0.01, max_depth=4, random_state=42)
actuarial_engine.fit(x_train_scaled, y_train)
final_predictions = actuarial_engine.predict(x_test_scaled)
print(f"Model Accuracy (R2 Score): {r2_score(y_test, final_predictions)*100:.2f}%")
print(f"Mean Absolute Error: ${mean_absolute_error(y_test, final_predictions):.2f}")

joblib.dump(actuarial_engine, 'medical_model.pkl')
joblib.dump(standard_scaler, 'scaler.pkl')
joblib.dump(X_features.columns.tolist(), 'features.pkl')
```

✓ 0.3s Python

```
Model Accuracy (R2 Score): 89.61%
Mean Absolute Error: $2549.68
```

7. Web Implementation & Deployment

The project was moved into a production-ready environment:

- **Backend:** Built with **Flask** to manage the model inference.
- **Serialization:** Used joblib to save the **XGBoost model** and the **StandardScaler** as .pkl files.
- **Interface Logic:** The website features a structured input form. For a professional user experience, the prediction result is rendered in a separate container only after the calculation is triggered.
- **Hosting:** Deployed on **Render** using a **Gunicorn** server to ensure the app stays live and handles requests efficiently.

8. Conclusion

By moving from **Random Forest (88%)** to **XGBoost (90%)**, we successfully built a model that can handle complex, non-linear medical data. The project demonstrates how technical model selection and proper outlier handling can significantly improve the reliability of AI-driven financial predictions.

Live Demo: https://medical-insurance-premium-predictor-70oe.onrender.com